BRACE Robot Framework

Developer Reference Guide

Protocol · Java Commands · Python Behaviors

1. System Overview

The BRACE framework consists of two communicating layers: a Java control program running on a host computer, and a Python server running directly on the MBot2's CyberPi. The host sends commands over TCP/IP, receives sensor telemetry over UDP, and uses the **`SensorCache`** to make decisions inside a state machine.

Java Host (Control)

- MBot2.java — command API
- **`RobotController`** — lifecycle & telemetry
- **`MazeRobot`** — state machine logic
- **`TcpRobotService`** — TCP transport
- **`SensorCache`** — live sensor data

Python Server (MBot2 CyberPi)

- COMMAND_HANDLERS registry
- **`BehaviorScheduler`** — threaded behaviors
- **`ResourceArbiter`** — hardware access control
- **`TelemetryStreamer`** — UDP sensor broadcast
- **`MBotServer`** — UDP discovery + TCP commands

2. Message Protocol

All messages are JSON objects serialized to a single line and terminated with a newline character (\n). The TCP server uses line-based framing — it accumulates bytes into a buffer and processes one complete JSON message per newline, which ensures that messages split across TCP packets are handled correctly.

2.1 Transport Channels

Channel	Port	Purpose

UDP Broadcast	9998	Robot discovery — host broadcasts DISCOVER, robot replies with ANNOUNCE
TCP Command	9990	Request/response command channel. Host sends COMMAND messages, robot replies with RESPONSE or ERROR
UDP Telemetry	9991	Robot streams sensor data to registered clients at ~100 Hz

2.2 Discovery Messages

Before connecting, the Java host broadcasts a DISCOVER message on all network interfaces. The robot responds with its IP address and command port.

Discovery Request (host → robot, UDP broadcast):

```
{
  "type": "DISCOVERY",
  "payload": {
    "action": "DISCOVER",
    "robot": "StingBot"
  }
}
```

Discovery Response (robot → host, UDP unicast):

```
{
  "type": "DISCOVERY",
  "id": "<echoed from request>",
  "payload": {
    "action": "ANNOUNCE",
    "robotType": "MBOT2",
    "robotId": "StingBot",
    "commandPort": 9990
  }
}
```

2.3 Command Message (host → robot, TCP)

Every command follows this envelope structure. The id field is a UUID generated by the Java codec, used to correlate the response.

```
{
  "type": "COMMAND",
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "payload": {
    "command": "COMMAND_NAME",
    "parameters": {
      "param1": value1,
      "param2": value2
    }
  }
}
```

2.4 Response Messages (robot → host, TCP)

Success response:

```
{
  "type": "RESPONSE",
  "id": "<echoed from command>",
  "payload": {
    "status": "OK",
    "message": "Human readable result",
    "data": { ... }
  }
}
```

Error response:

```
{
  "type": "ERROR",
  "id": "<echoed from command>",
  "payload": {
    "status": "ERROR",
    "errorCode": "ERROR_CODE_STRING",
    "message": "Error description"
  }
}
```

2.5 Telemetry Message (robot → host, UDP)

The robot broadcasts sensor readings at approximately 100 Hz to all registered UDP clients. Fields are only present if the corresponding sensor was successfully read in that cycle.

```
{
  "distance": 23.4,
  "line_offset": -0.5,
  "line_status": 3
}
```

Field	Type	Description
distance	float	Ultrasonic sensor distance in centimetres
line_offset	float	Lateral offset from line centre (negative = left of line)
line_status	int	Bitmask of line sensor readings. 0 = no line. Higher values indicate which sensors are over the line.

3. Built-in Commands Reference

The following commands are implemented in the Python server and can be called directly from the Java MBot2 API. Each command is described with its parameters and any data fields returned in the response.

Command	Key Parameters	Notes
MOVE_STRAIGHT	distance (float, cm)	Blocks until move completes. Negative distance moves backward.
MOVE_TIME	speed (float), time (float, s)	time < 0 moves indefinitely. Returns immediately for indefinite moves.
SET_MOTOR_POWER	left (float or "AS_IS"), right (float or "AS_IS")	Values −100 to 100. Use "AS_IS" to leave a motor unchanged.
TURN	degrees (int)	Positive = left, negative = right. Uses IMU closed-loop control.
MOVE_AND_TURN	speed, duration, diff, is_left (bool)	Arc movement. duration ≤ 0 runs indefinitely. diff controls arc radius.
STOP	(none)	Immediately halts motors at priority 100.
LED	status (ON/OFF/SHOW/MOVE), id, red, green, blue, color, step	Controls the five CyberPi LEDs.
CAMERA_LED	status (ON/OFF)	Toggles the smart camera illumination light.
GET_SENSOR	sensor (see below), light (YES/NO for camera)	Returns sensor data in the response data field.
TELEMETRY	action (REGISTER/DEREGISTER), port (int)	Registers the caller for UDP telemetry on the given port.
AVOID_CRASHING	threshold (float, cm, default 15)	Starts the AVOID_CRASHING background behavior.
STOP_AT_LINE	(none)	Starts the STOP_AT_LINE background behavior.
STOP_BEHAVIOR	behavior_name (string)	Signals one named behavior to stop on its next iteration.
STOP_ALL_BEHAVIORS	(none)	Signals all running behaviors to stop and halts motors.

GET_SENSOR sensor values:

sensor value	Response data field	Description
ULTRASONIC	distance_cm (float)	Distance from ultrasonic sensor in cm
LINE_STATUS	line_status (int)	Line sensor bitmask (same as telemetry)

LINE_OFFSET	line_offset (float)	Lateral offset from line centre
CAMERA_COLOR	color (string), x, y, area (int)	Color and position of detected object
CAMERA_LABEL	labels (int[])	List of detected label IDs (1–15)
IMU	yaw, pitch, roll (float)	Orientation angles in degrees

4. Adding a New Command on the Java Side

New commands are added to MBot2.java as public methods. The method encodes the command name and parameters into a JSON message, sends it over TCP via the **`execute()`** helper, and returns a **`CommandResult`** containing the response data. There is no registration step — any method that calls **`execute()`** with a recognized command name will work.

4.1 How execute() Works

The private **`execute()`** method in MBot2.java handles all protocol details. You do not need to construct JSON manually.

```
private CommandResult<JsonNode> execute(String command, Map<String,Object> params) {
    String json = codec.encodeCommand(command, params);  // builds JSON envelope
    service.send(json);                                   // sends over TCP + "\n"
    JsonResponse response = codec.decodeResponse(service.receive()); // blocks for
reply
    return new CommandResult<>(
        response.isOk(),
        response.getData(),
        !response.isOk() ? "ERROR("+response.getErrorCode()+"): " : "" +
response.getMessage()
    );
}
```

4.2 Step-by-Step: Adding a Command

Step 1 — Add a public method to MBot2.java

Use **`Map.of()`** to pass parameters. Use null if the command takes no parameters. The command name string must exactly match the @register_command name on the Python side.

```
/**
 * Example: plays a named sound clip on the robot.
 * @param clipName The name of the audio clip to play.
 */
public void playSound(String clipName) {
    execute(
        "PLAY_SOUND",
        Map.of("clip", clipName)
```

```
    );
}
```

Step 2 — If the command returns data, read it from the result

When the command returns data in the response, capture the **`CommandResult`** and extract the values from the **`JsonNode`**. Always check **`isSuccessful()`** before reading data.

```
/**
 * Example: reads the battery level from the robot.
 * @return Battery percentage (0-100), or -1 on failure.
 */
public int readBatteryLevel() {
    CommandResult<JsonNode> result = execute("GET_BATTERY", null);
    if (!result.isSuccessful()) {
        return -1;
    }
    return result.data().get("battery_pct").asInt();
}
```

4.3 Calling the New Method from MazeRobot

Once the method is added to MBot2.java, it is available through the mbot field in any **`RobotController`** subclass.

```
// Inside MazeRobot.run() or any state case:
mbot.playSound("yeah");

int battery = mbot.readBatteryLevel();
if (battery < 20) {
    System.out.println("Low battery: " + battery + "%");
}
```

> **Note:** Command methods in MBot2.java block until the robot sends a response. For long-running operations, prefer starting a behavior on the Python side and returning immediately, rather than keeping the TCP connection blocked for an extended period.

4.4 Reading Sensor Data — awaitNewData() and SensorSnapshot

Sensor data is delivered to the Java host by the UDP telemetry stream at approximately 100 Hz and stored internally in a **`SensorCache`**. Subclasses of **`RobotController`** must never read the **`SensorCache`** directly — it is private to **`RobotController`**. Instead, every iteration of the control loop calls **`awaitNewData()`**, which blocks until a fresh telemetry packet has arrived, atomically captures all sensor values into a **`SensorSnapshot`**, resets the new-data flag, and returns the snapshot to the caller.

SensorSnapshot fields:

Method	Type	Description
s.distance()	double	Ultrasonic sensor distance in centimetres
s.lineOffset()	double	Lateral offset from line centre (negative = left of line)
s.lineStatus()	int	Line sensor bitmask. 0 = no line detected.

How awaitNewData() works internally:

```
// In RobotController — called by subclasses, never overridden:
protected SensorSnapshot awaitNewData() {
    while (!sensors.hasNewData()) {
        Thread.sleep(10);           // yields until a telemetry packet arrives
    }
    return sensors.takeSnapshot();  // atomically resets flag and captures values
}
```

Correct usage pattern in run():

`awaitNewData()` must be called at the top of every loop iteration and its return value must be assigned to a local variable. The returned snapshot represents sensor state at the exact moment that iteration begins. Using a snapshot captured outside the loop, or calling **`awaitNewData()`** without assigning its return value, causes the state machine to act on stale data.

```
public void run() {
    currentState = RobotState.CRUISE;
    while (currentState != RobotState.STOP) {

        SensorSnapshot s = awaitNewData();// CORRECT — fresh snapshot every iteration
        switch (currentState) {
            case CRUISE:
                // chill out!
                break;
            case AHHHH:
                // Don’t freak out!
                // use s.distance()
                break;
            // ... other cases use s.distance(), s.lineStatus(), s.lineOffset()
        }
    }
}
```

Warning: Do not call **`awaitNewData()`** without assigning its return value, and do not capture the snapshot before the loop and reuse it across iterations. Both mistakes cause the state machine to act on stale sensor readings that no longer reflect the robot's current environment. Every iteration must call **`awaitNewData()`** and assign the result to a fresh local variable inside the loop.

5. Adding Commands and Behaviors on the Python Side

The Python server uses a decorator-based registry. Any function decorated with `@register_command("NAME")` is automatically available as a command. Behaviors are background functions managed by the `BehaviorScheduler` that loop until signaled to stop.

5.1 The ResourceArbiter and Priority System

All hardware resources (motors, sensors, LEDs) are protected by the `ResourceArbiter`. Every behavior and command handler that touches hardware must acquire the relevant resource before use and release it in a finally block. This prevents two threads from issuing conflicting hardware instructions simultaneously.

Priority reference table:

Priority Range	Category	Who uses it
250	Safety-critical	AVOID_CRASHING — must never be blocked
180	Navigation stop	STOP_AT_LINE, STOP_ALL_BEHAVIORS handlers
100–150	Deliberate maneuvers	AVOID_OBJECT, MOVE_OBJECT, STOP command
50–80	Direct CMD handlers	MOVE_STRAIGHT, TURN, MOVE_AND_TURN, GET_SENSOR
30–40	Sensor behaviors	FOLLOW_LINE line read, CMD sensor reads
10	Background telemetry	Telemetry streamer — lowest priority, non-blocking

Important: The arbiter uses >= to decide preemption: a new `acquire()` succeeds if its priority is greater than or equal to the current owner's priority. Therefore, strictly distinct priorities must be used for behaviors that should never preempt each other. Two behaviors at the same priority will evict each other on every loop iteration.

5.2 Adding a Simple (Instant) Command

An instant command executes its logic synchronously, sends a response, and returns. It is appropriate for actions that complete quickly — setting LEDs, playing a sound, reading a sensor value.

All additions should be placed in the Extension Zone at the bottom of server-mbot-registry.py.

Template:

```
@register_command("MY_COMMAND")
def handle_my_command(payload):
    params = payload.get("parameters", {})
    # Extract parameters
    my_param = params.get("my_param", default_value)

    # Acquire the hardware resource you need
    if arbiter.acquire("motors", "MY_CMD", 50):
        try:
            # Do hardware work here
            mbot2.straight(my_param)
            return ok_response("Done")
        finally:
            arbiter.release("motors", "MY_CMD")

    # Return error if resource could not be acquired
    return error_response("RESOURCE_BUSY", "Motors are busy")
```

Example — play a sound clip:

```
@register_command("PLAY_SOUND")
def handle_play_sound(payload):
    params = payload.get("parameters", {})
    clip = params.get("clip", "yeah")
    cyberpi.audio.play(clip)
    return ok_response("Playing " + clip)
```

Example — read battery level and return data:

```
@register_command("GET_BATTERY")
def handle_get_battery(payload):
    pct = cyberpi.get_battery_level()
    return ok_response("Battery level", {"battery_pct": pct})
```

5.3 Adding a Background Behavior

A behavior is a function that the **`BehaviorScheduler`** calls repeatedly in its own thread until a stop flag is set. Behaviors are suitable for continuous monitoring tasks — following a line, watching the ultrasonic sensor, reacting to the environment.

Key rules for behavior functions:

- Must complete quickly — the scheduler calls the function in a tight loop with a 20 ms sleep between calls. Long-blocking code will delay all other behaviors running in the same thread.
- Must use non-blocking arbiter acquires (blocking=False) for motor resources, so a higher-priority behavior is never delayed.
- Must release all resources in finally blocks, even if an exception occurs.
- Should be idempotent — it may be called hundreds of times per second.

Template:

```
def my_behavior(param1, param2):
    # Step 1: Acquire sensor resource (non-blocking for sensors is fine)
    if arbiter.acquire("my_sensor", "MY_BHV", priority):
        try:
            value = read_sensor_here()
        finally:
            arbiter.release("my_sensor", "MY_BHV")

    # Step 2: Early exit if no action needed
    if not action_needed(value):
        return

    # Step 3: Acquire motor resource (non-blocking — respect priority)
    if arbiter.acquire("motors", "MY_BHV", priority, blocking=False):
        try:
            # Issue motor commands here
            mbot2.drive_speed(em1, em2)
        finally:
            arbiter.release("motors", "MY_BHV")


@register_command("MY_BEHAVIOR")
def handle_my_behavior(payload):
    params = payload.get("parameters", {})
    param1 = params.get("param1", default1)
    scheduler.start_behavior("MY_BEHAVIOR", my_behavior, param1, param2)
    return ok_response("MY_BEHAVIOR started")
```

Example — flash LEDs when battery is low:

```
def low_battery_alert_behavior():
    pct = cyberpi.get_battery_level()
    if pct > 20:
        return   # nothing to do
    if arbiter.acquire("led", "BATT_ALERT", 60, blocking=False):
        try:
            cyberpi.led.show("r k r k r")   # flash red
        finally:
            arbiter.release("led", "BATT_ALERT")


@register_command("MONITOR_BATTERY")
def handle_monitor_battery(payload):
    scheduler.start_behavior("MONITOR_BATTERY", low_battery_alert_behavior)
    return ok_response("Battery monitor started")
```

5.4 Stopping a Behavior

Behaviors are stopped by name. The scheduler sets their stop flag and they exit on the next iteration. This is already wired up — you do not need any special teardown code in most behaviors.

```
# From Java (via MBot2.java):
mbot.stopBehavior("MONITOR_BATTERY");
```

```
mbot.stopAllBehaviors();

# Directly on the Python side:
scheduler.stop_behavior("MONITOR_BATTERY")
scheduler.stop_all()
```

Note: **`stop_behavior()`** and **`stop_all()`** are asynchronous — they signal the behavior to stop but do not wait for it to finish. If hardware must be in a known state immediately after stopping, call **`mbot2.forward(speed=0)`** explicitly after **`stop_all()`**.

6. End-to-End Walkthrough: Adding a Flashlight Command

This section shows the complete set of changes needed to add a new command — TOGGLE_FLASHLIGHT — that turns the camera LED on or off by name, callable from Java.

Step 1 — Add the Python handler (server-mbot-registry.py)

```
# In the Extension Zone at the bottom of server-mbot-registry.py

@register_command("TOGGLE_FLASHLIGHT")
def handle_toggle_flashlight(payload):
    params = payload.get("parameters", {})
    state = params.get("state", "OFF")   # "ON" or "OFF"
    if arbiter.acquire("camera", "FLASHLIGHT", 30):
        try:
            if state == "ON":
                mbuild.smart_camera.open_light()
            else:
                mbuild.smart_camera.close_light()
            return ok_response("Flashlight " + state)
        finally:
            arbiter.release("camera", "FLASHLIGHT")
    return error_response("RESOURCE_BUSY", "Camera in use")
```

Step 2 — Add the Java method (MBot2.java)

```
/**
 * Toggles the camera flashlight on or off.
 * @param on true to turn the light on, false to turn it off.
 */
public void setFlashlight(boolean on) {
    execute(
        "TOGGLE_FLASHLIGHT",
        Map.of("state", on ? "ON" : "OFF")
    );
}
```

Step 3 — Call it from MazeRobot

```
// Turn light on before scanning, off after
mbot.setFlashlight(true);
String color = mbot.getColorObjectFromCamera(false);  // light already on
mbot.setFlashlight(false);
```

7. Quick Reference

Response Helper Functions (Python)

Function	Usage
`ok_response(msg)`	Return a success response with no data
`ok_response(msg, {"key": val})`	Return a success response with a data payload
`error_response("CODE", "msg")`	Return an error response with an error code and message

Arbiter Resource Names

Resource Name	What it Guards
motors	All motor/drive commands (drive_speed, straight, forward, EM_set_power)
ultrasonic	mbuild.ultrasonic2.get()
line	mbuild.quad_rgb_sensor reads (get_line_sta, get_offset_track)
camera	mbuild.smart_camera (detect, learn, open_light, close_light)
led	cyberpi.led (on, off, show, move)
imu	cyberpi.get_yaw() and related IMU reads

CommandResult<T> Fields (Java)

Field	Description
isSuccessful()	true if the robot returned status OK
data()	JsonNode containing the response data payload (may be null)
message()	Human-readable result or error message string

BRACE Framework — Developer Reference